

SemDiv论文复现笔记：语义多样队友生成与开放协作

SemDiv论文复现笔记：语义多样队友生成与开放协作

SemDiv 论文复现笔记：语义多样队友生成与开放协作

论文：LLM-Assisted Semantically Diverse Teammate Generation for Efficient Multi-agent Coordination

作者：Lihe Li, Lei Yuan, Pengsen Liu, Tao Jiang, Yang Yu (ICML 2025)

论文来源：随文归档的公开论文 PDF

官方代码：<https://github.com/lilh76/SemDiv>

公开版说明：本文仅基于论文与公开代码整理，不包含访问凭据、服务地址、运行设备信息、目录信息或未公开实验数据。

一、这篇论文解决什么问题？

传统多智能体强化学习通常在固定队友组成下训练和评测；但在开放协作中，受控智能体会遇到行为未知、不可控制的队友。只用随机种子或策略差异训练出的队友虽然多样，却没有人能读懂的“行为含义”，因此既难以高效覆盖队友空间，也无法在部署前根据队友的一句话偏好直接适配。

SemDiv 的核心主张是：先在**语言/语义空间**探索“如何协作”，再把每条协作偏好落地为可训练的奖励函数和队友策略。最终受控智能体保存一组专门对应不同语义的策略头；面对新队友时，LLM 根据新队友提供的自然语言偏好选择最合适的头，而不是先通过试错互动识别对方。

最简流程如下：



|
v

为受控智能体新建策略头 h_k ，持续学习与该队友协作

|
v

新队友描述 $b \rightarrow$ LLM 选择 $k \rightarrow$ 使用共享特征提取器 + h_k 执行

复现目标建议分三级：

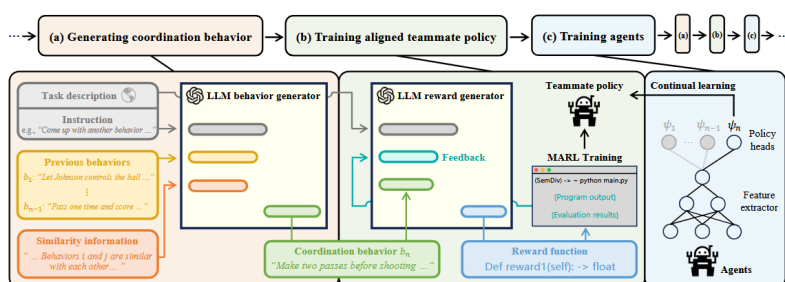
1. **最小跑通**：固定一条人工编写的额外奖励，完成“训练队友 - 训练 ego - 双模型评测”闭环，不调用 LLM。
2. **方法复现**：接入 LLM，得到 6 个通过验证的队友，并核对行为库、候选策略、策略头数是否一致。
3. **结果复现**：三随机种子，按论文协议在每个环境的 5 个未知代表性队友上报告任务成功率 R1 和偏好满足率 R2。

读完本文应回答的四个问题

1. 为什么“策略数值上不同”不等于“协作语义上不同”？
2. 一句自然语言偏好如何被转成可以训练队友的奖励信号？
3. 怎样证明一个新队友既能完成任务、又实现了指定行为、还没有和已有队友重复？
4. 遇到未见队友时，为什么多头策略加语言选择可以少走试错步骤？

二、问题定义与方法拆解

论文将任务建模为含受控智能体集合 N_{ag} 与不可控队友集合 N_{tm} 的 Dec-POMDP。执行前，队友给出自然语言描述 b ，例如“射门前希望先完成一次传球”。受控智能体要最大化与未知队友的期望折扣回报，同时尽量满足 b 所描述的协作偏好。



1. 语义行为的迭代生成

第 k 轮前的库为 $P_{(k-1)} = \{(b_j, pi_{tm,j}, l_j)\}$ ： b_j 是行为描述， l_j 表示对应队友是否通过验证。行为生成器的提示由任务说明、生成指令、此前有效行为，以及“哪些不同描述实际产生了相似策略”的反馈组成。这样 LLM 不只被要求“再想一个不同的策略”，而是被要求提出一个具体、可实现、且避开已知重复区域的协作方案。

2. 从行为描述到队友策略

给定 b_k ，第二个 LLM 生成可执行的额外奖励函数 $r_{\hat{k}}$ 。候选队友与其自博弈搭档共同训练，优化目标为：

Plaintext

$$J(\pi_{ag_tilde}, \pi_{tm,k}) = E[\sum_t \gamma^t (r_t + \lambda_2 * r_{\hat{k},t})]$$

其中原始环境奖励 r_t 保证完成任务； λ_2 从 1 衰减到 0，避免智能体只钻额外奖励的空子、却忘了完成任务。

候选队友需要经过五类检查：奖励函数是否可运行、是否完成原任务、额外奖励是否真的在变化、轨迹是否与 b_k 对齐、是否与已有队友重复。论文的“策略相似”不是比较参数距离，而是检验一个能与旧队友协作的 agent 是否也能与新队友协作；若两种原始/额外奖励下的相对回报差均不足阈值 ϵ ，则视为重复。

3. 持续学习的多头协调策略

单一策略很难同时满足互相冲突的协作偏好，顺序训练还会遗忘旧队友。故第 k 个有效队友对应一个新头 $h_{\psi,k}$ ：

Plaintext

$$\pi_i(o_i) = h_{\psi,i,k}(f_{\phi,i}(o_i))$$

旧头冻结，只更新共享特征 f_{ϕ} 与新头。为了保护旧知识，论文对共享特征相对上一轮快照加 L_p 正则：

Plaintext

$$\max J(\pi_{ag}, \pi_{tm,k}) - \alpha / |N_{ag}| * \sum_i ||\phi_i - \bar{\phi}_i||_p$$

执行时，LLM 读取任务说明、新队友描述和已学习行为列表，输出策略头编号 k ；之后不再需要试错式的在线适应。

4. 一轮 SemDiv 的端到端算法

下面的伪代码刻画了论文方法的因果顺序。它也可作为团队阅读代码时的主索引。

Plaintext

输入：任务描述 desc，目标有效队友数 K
初始化：有效库 $P = \text{空}$ ，策略头集合 $H = \text{空}$

重复直到 $|P| = K$ ：

1. LLM(desc, 已有效行为, 相似性反馈) -> 新行为 b

2. LLM(desc, b, 既往失败反馈) -> 额外奖励 $r_{\hat{}}$

3. 用 $r + \lambda * r_{\hat{}}$ 训练候选队友 π_{tm}

4. 验证 π_{tm} : 可运行、完成任务、奖励非退化、轨迹对齐、策略非重复

5. 若验证失败: 把失败原因回填给 LLM, 重试或放弃 b

6. 若验证成功: $P \leftarrow P \cup \{(b, \pi_{tm})\}$
新建策略头 $h_{|P|}$, 冻结旧头; 训练共享表征和 $h_{|P|}$ 与 π_{tm} 协作
- 执行: LLM(未知队友描述, P 中的行为文本) -> k ; 使用第 k 个策略头

这里有两个闭环。外层闭环通过“相似/失败反馈”让语言探索越来越贴近可实现且未覆盖的协作行为；内层闭环通过额外奖励把行为文本变成队友轨迹。只保留其中任一个，方法都会退化：没有外层闭环，LLM 会重复提出看似不同但策略相近的偏好；没有内层闭环，行为文本无法转化为可供 ego 学习的交互对象。

5. 关键概念辨析

概念	在 SemDiv 中的含义	不应误解为
语义多样性	协作意图在自然语言层面可区分，且落地后带来不同交互模式	仅是网络参数或轨迹像素不同
额外奖励	把偏好转为可优化的训练信号	替代环境原始任务奖励
队友有效性	同时完成任务、对齐目标、与历史队友有区分	单次回报高即可接受
多头策略	共享任务知识、为冲突偏好保留专门决策出口	为每个队友完全重新训练网络
语言选择	执行前基于描述路由到策略头	运行中持续从文本直接生成动作

三、论文实验结论

环境、算法与指标

论文覆盖四个环境：Level-Based Foraging（LBF）、改造的 Predator-Prey（PP）、SMACv2 和 Google Research Football（GRF）。LBF、PP、SMACv2 使用 VDN；GRF 使用 MAPPO；行为生成、奖励生成和头选择均使用 GPT-4o-2024-08-06。

每个方法生成 6 个训练队友；每个队友取初、中、末三个 checkpoint，因此用于 agent 训练的总队友策略数为 18。评测面对每环境 5 个训练期间完全未知、且具有代表性偏好的队友，使用三随机种子：

- R1：任务完成成功率。
- R2：满足队友偏好行为的成功率。

主结果（论文表 1）

方法	平均 R1	平均 R2	解释
Oracle（测试队友自博弈）	0.95	0.95	非公平上界
SemDiv	0.74	0.68	LLM 选择多头策略
SemDiv-R1	0.81	0.69	事后知道每个测试对象最佳 R1 的上界
SemDiv-R2	0.78	0.73	事后知道最佳 R2 的上界
SemDiv-PBT	0.63	0.49	同一批 SemDiv 队友，但用两阶段单策略训练
Macop-R2	0.60	0.49	迭代生成但非语义探索
MEP	0.58	0.42	策略多样性基线
FCP	0.48	0.34	不同随机种子人口
LLM-Agent	0.52	0.47	仅 LLM、无任务 RL

相对最强基线 Macop，SemDiv 的平均 R1/R2 分别提升约 19%/39%。其中最值得注意的对照不是“SemDiv 比 FCP 高”，而是：

- 1. SemDiv-PBT > FCP/MEP/LIPO：语义引导先提升了训练队友的覆盖质量。
- 2. SemDiv > SemDiv-PBT：仅有多样队友不够；多头结构处理了不同偏好间的多模态冲突。
- 3. SemDiv-Dist 明显更差（平均 R1=0.42，R2=0.33）：用通用文本嵌入的最近邻替代 LLM 推理，不能可靠完成“描述到策略头”的映射。

分环境结果与读法

环境	SemDiv R1	SemDiv R2	该环境主要检验什么
LBF	0.90 +/- 0.05	0.90 +/- 0.05	采集目标与位置偏好的配合
PP	0.72 +/- 0.03	0.54 +/- 0.10	围捕高价值目标与独自捕捉低价值目标的取舍
SMACv2	0.65 +/- 0.02	0.64 +/- 0.02	随机化战场中目标选择与集火协同

GRF	0.67 +/- 0.08	0.62 +/- 0.07	传球次数、射门者等长程协作战术
-----	---------------	---------------	-----------------

R1 与 R2 必须联合解读。只看 R1 会把“虽然完成任务、但违背队友偏好”的策略误判为成功；只看 R2 则可能接受“完成了指定动作、但没有完成任务”的策略。PP 中 R2 低于 R1 尤其说明，任务成功和偏好满足并非天然一致，需要额外奖励和头选择共同约束。

GRF 案例最能展示论文的强项：语义探索得到的队友覆盖“零次传球、一次传球、两次传球”等战术，FCP 一类只依赖随机初始化的方法较难稳定发现多次传球。论文再以成功轨迹的表示进行可视化，显示 SemDiv 队友的覆盖更分散。这是“行为空间覆盖扩大”的辅助证据，不是代替 R1/R2 的最终指标。

基线应该如何比较

基线类别	代表方法	与 SemDiv 的核心差异
随机人口	FCP	用不同随机种子制造差异，没有显式语义目标
多样性正则	MEP、LIPO	在策略层压低相似性，但不生成可解释协作意图
迭代反兼容	Macop	根据与现有 agent 的不兼容性找新队友，缺少语言层的行为发现与路由
两阶段训练	SemDiv-PBT、Macop-PBT	用队友池训练单一策略，不能消除多偏好冲突
文本嵌入路由	SemDiv-Dist	以描述距离替代 LLM 推理，无法理解任务约束和互补关系
纯 LLM 决策	LLM-Agent	不借助环境交互训练，复杂任务中缺少任务特定技能

为公平比较，论文统一以 6 个队友及其初、中、末训练阶段的策略参与后续 agent 训练。这一点决定了复现时不能只比较“生成了多少个行为文本”，还必须比较可用于训练的队友策略数量和采样阶段。

四、实现模块与论文对应

实现可分为四层：行为与奖励生成层、候选队友训练与验证层、多头持续学习层、自然语言头选择与评测层。LBF、PP、SMACv2 使用基于值分解的训练实现，GRF 使用基于策略梯度的训练实现。

与论文的关键对应关系：

- 行为生成器将成功样例和“可行但重复”的样例回填至后续提示，从而迭代修正语义探索方向。
- 奖励生成器把自然语言偏好转成环境可调用的额外奖励，并将其交给现有 MARL 训练器学习候选队友。
- 验证器依次检查运行有效性、原始回报、额外回报、轨迹对齐和与历史队友的相似性；实验产物应记录为可审计的行为库。
- 多头网络由共享特征提取器和独立策略头组成。值分解实现使用三层 MLP 加 GRU 作为共享特征，策略头为独立 MLP。
- 持续学习阶段仅对共享特征相对上一轮快照施加平方 L2 正则，以缓解遗忘；评测阶段由 LLM 在有限候选头中输出一个编号。

多头与持续学习的实现含义

新头负责吸收“这一个队友偏好下要如何行动”的差异，共享特征负责保留“这个任务中什么状态、动作历史和局部观测重要”的共性。旧头被冻结，因此旧的输出映射不会被直接覆盖；但共享特征仍会变化，旧头的输入分布也可能漂移。这就是论文还要约束共享特征参数变化的原因。

从复现角度，应分别记录三类指标：新队友训练回报、旧队友上的回放回报、共享特征正则项。只记录新队友回报无法发现灾难性遗忘；只记录正则项也无法说明旧头仍然可用。若旧任务表现急剧下降，优先检查特征快照是否在正确时机保存、旧头是否真的冻结、以及正则系数是否生效。

五、如何复现

当前复现状态

本笔记目前完成了论文阅读与公开代码审计，尚未在目标环境中执行完整训练。因此，文中“源码确认”表示可以直接从实现读出的流程与门槛；“建议”表示推荐的复现实验设计；论文表格中的数值来自论文，不是本次复现产出。后续团队成员应在运行总览记录中持续更新状态，避免将设计记录、训练日志和最终结论混为一谈。

1. 复现范围与推荐顺序

应把“复现”拆成四件不同的事，而不是只问最终数字是否接近论文。

层级	目标	成功标准	失败时先查什么
工程可运行	环境、训练器和日志能闭环	能完成一个短回合的 reset、step、反向传播、保存与加载	依赖版本、环境 API、观测/动作维度
行为可落地	额外奖励能塑造目标行为	原始任务和额外奖励都不是退化信号	奖励触发条件、状态字段、奖励尺度
方法可成立			

	生成、筛选、多头持续学习均按论文逻辑工作	得到若干有效且可区分的队友，旧头不显著遗忘	验证门槛、头冻结、特征正则
结果可比较	对未知队友的泛化接近论文趋势	三随机种子下 R1/R2 与消融关系方向一致	训练预算、评测队友、头选择、随机性

建议从 LBF 开始。它依赖较少，能够先验证“行为文本 - 奖励 - 队友 - 策略头 - 评测”的完整链路；PP 用于检查目标偏好冲突，SMACv2 用于检查随机化战场中的协作，GRF 最后用于验证长程战术与传球偏好。不要在基础闭环尚未稳定时直接进入 GRF。

本仓库的自动化默认目标是收集 6 个有效队友；每个候选行为最多允许两次奖励函数尝试，总候选行为最多 30 个。这些上限意味着“6 个有效队友”不是 6 次训练，而是一段包含生成、失败重试、筛选和持续学习的实验过程。

2. 开始前的实验冻结

开始任何训练前，先建立一份不可修改的实验卡。它至少固定论文版本、公开代码版本、依赖版本、环境任务定义、随机种子集合、模型服务版本、行为验证规则和评测协议。发生错误时可以创建新实验卡，但不要在同一实验编号下悄悄替换奖励、种子或依赖。

建议先做三个不依赖 LLM 的烟雾测试：

- 1. 使用零额外奖励训练极短回合，确认原始任务回报、动作可用性和终止标记正常。
- 2. 使用人工写出的简单事件奖励，确认额外奖励会被环境返回、记录并进入训练曲线。
- 3. 用保存的队友模型训练或评测一个 ego 头，确认检查点恢复后行为不变。

三项都通过后，才开启“LLM 生成奖励”的变量。这样一旦失败，就能判断是环境接口问题、奖励实现问题，还是 LLM 生成质量问题。

3. 源码改造清单：改什么、为什么改、如何验收

所有改动应在公开代码的实验副本中完成，并为每个实验卡保存补丁或变更说明。不要把“为本地跑通而作的改动”与“原仓库行为”混在同一结果表中。下表中的“严格复现”是尽量保持发布实现；“方法复现”是为验证论文主张而进行的透明改造。

优先级	文件与位置	必要修改	影响范围
必须	language/call_llm.py 的 LLM.__init__()	从安全的运行时环境读取访问凭据，禁止在源码中填写	仅连接方式，不应改变算法
必须	language/semdiv.py 的 llm_write_code()	生成代码写入前做语法检查、保存副本、限制解析重试次数	提高可审计性和失败可见性

建议	language/semdiv.py 的复制与子进程逻辑	用 Python 标准库替代平台相关的复制、命令执行和进程检测	改善可移植性，不应改变训练目标
建议	pymarl/src/run_contiual.py 的 get_timestep()	显式记录初/中/末检查点，移除对命名格式的隐式假设	影响训练人口构成，必须记录
调研项	pymarl/src/learners/multi_head_learner.py 的 mixer 初始化	决定保留发布实现，还是启用 VDN mixer 做论文一致性实验	会改变算法，不能与原实现混报
建议	pymarl/src/modules/agents/multi_head_agent.py 的 forward()	用有信息量的异常替代 assert 0	仅改善定位能力

3.1 模型服务访问：language/call_llm.py

原实现将空字符串放在 `api_key_list` 中，且使用早期 SDK 的 `openai.ChatCompletion.create` 调用。至少应把凭据从源码移出。若保持早期 SDK，修改 `import` 区和 `LLM.__init__()` 为下列模式：

```
Python
import os

api_key = os.environ.get("OPENAI_API_KEY")
if not api_key:
    raise RuntimeError("OPENAI_API_KEY is not configured")
api_key_list = [api_key]
```

这段替换原先的空 `api_key_list` 即可。不要把真实凭据写入实验卡、生成日志、截图或版本控制记录。

若依赖使用当前 SDK，则不能只替换一行调用。应将 `gpt_agent` 改为持有客户端，并同步替换响应读取和异常处理：

```
Python
from openai import OpenAI

self.client = OpenAI(api_key=api_key)
response = self.client.chat.completions.create(
    model=self.model_name,
    messages=[{"role": "user", "content": question}],
    temperature=temperature,
    stop=stop,
```

```
)  
return response.choices[0].message.content
```

二者只能选其一：要么固定早期 SDK 并保留旧异常类型，要么完整迁移客户端、异常处理和重试逻辑。半迁移最常见的信号是生成阶段立刻报 ChatCompletion 接口错误。验收方式是在不启动 MARL 训练的情况下，对固定提示执行一次大模型和一次小模型调用，并保存**不含凭据**的模型名、响应状态和重试次数。

3.2 生成奖励的安全接入：language/semdiv.py 的 llm_write_code()

原函数会解析 LLM 输出，将 additional_reward 改名后直接追加到环境实现，并用裸 except: pass 吞掉所有异常。这会导致两种难定位问题：解析格式稍有变化就无限重试；生成内容有语法错误时，环境文件可能已被污染。

在写入环境前加入以下两步。第一步验证解析出的函数本身可编译；第二步把写入前的文件复制到本轮实验归档中：

```
Python  
# store_code 已从 LLM 输出中抽取完成后  
compile(store_code, f"<generated_reward_{self.tm_idx}>", "exec")  
  
backup_path = os.path.join(  
    self.lib_dir, f"env_before_tm_{self.tm_idx}_attempt_{self.n_attempt_behavior}.py"  
)  
shutil.copy2(self.env_file, backup_path)  
  
with open(self.env_file, "a", encoding="utf-8") as f:  
    f.write(textwrap.indent(write_code, "  "))
```

并将裸异常改为带上限的显式失败。例如，将 while True 改为有限次数循环；解析失败时记录原始输出的哈希、异常类型和尝试编号，达到上限则写入 bug 状态，而不是无声重试。compile() 只能排除 Python 语法错误，不能发现未定义的环境属性，所以仍要保留下一步的单步调用测试和短训练验证。

3.3 环境选择与运行归档：language/semdiv.py 的主入口

主入口以注释切换 LBF、PP、SMACv2 与 GRF。建议将其改成显式配置，并对未知值立即报错：

```
Python  
env = os.environ.get("SEMDIV_ENV", "lbf")  
supported_envs = {"lbf", "pp", "sc2"}  
if env not in supported_envs and not env.startswith("football:"):   
    raise ValueError(f"Unsupported environment: {env}")
```

这样环境选择会进入运行记录，避免在同一个实验名称下手工改注释导致结果来源不明。原实现还使用外部命令复制运行文件、启动训练并查询进程。建议同时改两处复制操作为 `shutil.copy2`，把训练启动改为 `subprocess.Popen`，并保存返回的 `Popen` 对象，通过 `process.poll()` 判断完成。不要只替换启动命令而保留文本匹配式的进程检测，否则失败恢复仍不可靠。

3.4 多头训练的检查点选择： `pymarl/src/run_continual.py` 的 `get_timestep()`

原实现对每个队友选取初、中、末三个检查点，其中“中间”检查点通过 TensorBoard 原始回报曲线确定；路径推断中还包含对命名格式和年份字符串的假设。更具体地说，代码计算的是 $(\text{末回报} - \text{初回报}) / 2$ ，再寻找曲线上最接近该数值的点；这并不是通常的回报算术 midpoint $(\text{末回报} + \text{初回报}) / 2$ 。应将实际选择的三个训练步保存为明确配置，而不是让后续运行重新推断。推荐增加一个配置项：

```
Python
# 伪代码：在加载队友模型前固定并写入 config
selected_steps = {
    "early": min(timesteps),
    "middle": chosen_middle_step,
    "late": max(timesteps),
}
json.dump(selected_steps, open("selected_checkpoints.json", "w"))
```

首次严格复现时，可以保留原始的中间检查点规则，同时把 `selected_steps` 作为输出保存；后续消融实验才允许切换为按训练步数居中或固定比例选取。验收时检查：每个队友确实有三个不同的可加载检查点、每个检查点的原始回报被记录、训练人口大小等于队友数乘以每队友检查点数。

3.5 VDN/mixer 的实现分叉： `pymarl/src/learners/multi_head_learner.py`

当前 learner 初始化中有一行：

```
Python
args.mixer = None
```

它会使随后创建 `VDNMixer` 或 `QMixer` 的分支不可达。这里不能不加说明地“修好”：

- **复现发布实现：**保留该行，在记录中注明多头训练实际未启用 mixer。
- **检验论文中 VDN 设定：**删除或注释该行，让配置中的 `mixer: vdn` 生效；另起实验编号，不能与前者合并平均。

启用 VDN 后，启动时应记录 `learner.mixer` 的实际类型，并检查训练损失从逐智能体 TD 误差变为经 mixer 聚合的团队值。若只改配置文件、不改这一行，算法仍然不会使用 VDN。

3.6 策略头索引的调试： `pymarl/src/modules/agents/multi_head_agent.py`

`forward()` 在 `head_id` 不存在时打印编号后执行 `assert 0`。改成明确异常，可直接暴露“队友编号、头编号和检查点数量不一致”的根因：

Python

```
if head_id is None or head_id not in self.head_dict:
    available = sorted(self.head_dict)
    raise KeyError(f"Unknown head_id={head_id}; available={available}")
q = self.head_dict[head_id](h)
```

完成修改后，至少执行三项检查：首轮训练能访问第一个头；载入上一轮 ego 后旧头仍存在；给定越界编号时会得到上述明确错误而不是静默训练错误。

4. 阶段 A：人工奖励的最小闭环

先选择一个**可观察且一次回合内可判定**的行为，例如“协作采集指定类别目标”或“在满足任务前不触发某一无关动作”。人工奖励应满足三个条件：触发条件只依赖环境已有状态；正常完成原任务时不与原始奖励冲突；未触发时返回零或小值而非持续惩罚。

最小闭环按以下顺序执行：

Plaintext

固定人工行为描述

- > 编写额外奖励并对单步状态做手工检查
- > 训练候选队友
- > 同时检查原始回报与额外回报
- > 保存队友模型并创建第一个 ego 头
- > 用该头与同一队友评测
- > 重载模型后重复评测，比较关键指标

通过标准不是“额外奖励很高”。一个可信的最小闭环需要同时满足：原始任务能够完成；额外奖励在目标事件发生时才有响应；轨迹中能看到预期行为；模型重载后评测稳定。若额外奖励升高而原始回报长期很低，优先检查是否存在 reward hacking；若原始回报正常而额外奖励恒定，优先检查奖励函数是否真正接入环境返回值。

5. 阶段 B：接入 LLM 的候选队友生成

启用 LLM 后，每个候选行为必须经历“文本 - 代码 - 训练 - 验证”的闭环。行为文本应写成可判断事件，而不是泛泛偏好。好的例子是“完成任务前先与队友在指定目标处会合”；不好的例子是“更积极地合作”。前者可以在轨迹中判断，后者无法稳定转成奖励。

奖励生成后，先做静态审查，再训练：

- 函数签名是否符合环境调用约定，引用的状态属性和辅助函数是否存在。

- 目标事件是否在一个回合内可能发生；若事件极少发生，训练信号会长期为零。
- 额外奖励的最大量级是否远大于原始任务奖励；若是，先缩放或分段奖励。
- 奖励是否只依赖最终成功事件；必要时为中间可观察进展提供稠密、但不改变目标含义的信号。

源码会对候选队友分配状态。团队应按下表解释它们，并在记录中保留每次状态转换的原因：

状态	含义	复现处理建议
bug	训练或奖励执行异常	保存错误栈和奖励版本；先做最小单步复现
failed	原始任务表现低于环境要求	检查额外奖励是否压制任务奖励，或任务是否过难
constant	额外奖励在训练前后近似不变	检查事件可达性、状态更新时机和数值尺度
misaligned	轨迹经行为核验后不符合文本	对照轨迹事件和奖励触发日志，修改行为或奖励
similar	与已有队友在兼容性检查中无法区分	将相似对象与原因回填给下一次行为生成
success	通过上述门槛	固化行为、奖励、模型和验证证据，再训练新头

注意：失败不是应被删除的噪声。它是 SemDiv 外层探索循环的训练数据。尤其是 similar 的语义反馈，会帮助后续行为生成器避免用不同措辞重复同一种策略。

源码审计还显示，候选队友的原始任务门槛采用按环境区分的容忍度：LBF 为 0.3，PP 为 0.6，SMACv2 与 GRF 为 0.5；也就是候选模型的近期原始回报若低于对应基准回报乘以 (1 - 容忍度)，会被标为 failed。这些值是当前实现的筛选规则，不等同于论文主结果中的评价指标。复现时应原样记录使用的门槛，并单独测试门槛变化对“有效队友数”和最终 R1/R2 的影响。

6. 阶段 C：持续学习与多头策略训练

每得到一个有效队友，按“加载上一轮 ego - 保存共享特征快照 - 新建/重置当前头 - 冻结旧头 - 训练当前头与共享特征 - 评测新旧队友”的顺序进行。代码中的持续学习正则约束的是共享特征层，而不是策略头；因此旧头是否保存、快照是否建立、正则项是否非零，都是必须记录的检查点。

推荐在每一轮后构造一个兼容性矩阵：行是策略头，列是已验证队友，单元格同时记录原始任务表现和偏好满足表现。理想图形接近对角线较高：第 k 个头擅长第 k 类偏好，但旧头仍保持各自领域性能。若所有头的表现几乎相同，可能是头没有正确分离；若新头变好而旧列整体变差，可能出现遗忘或共享特征漂移。

需要特别区分“加载的队友人口”和“实际用于更新的队友”。当前实现会为每个有效队友取初、中、末三个检查点，加载它们并在测试时遍历；但更新 ego 时只从**最新队友**的三个检查点采样。这与“每一步都均匀回放所有历史队友”不同，也是论文持续学习设定的一部分。实验记录应分别写出：已加载检查点数、评测覆盖的历史队友数、实际训练采样的队友集合。否则很容易把“评测了历史队友”误写成“用历史队友共同训练”。

中间检查点也不是简单按训练步数二分。实现会根据原始任务回报曲线寻找最接近 (末回报 - 初回报) / 2 的点；这与通常的起末回报中点不同。该规则依赖日志完整性和曲线读取逻辑；复现时应把每个队友实际选取的初、中、末检查点及其回报写入记录，避免不同检查点选择规则造成不可见差异。

持续学习部分还有一个源码核查点：多头 learner 中显式关闭了 mixer，即便配置中出现值分解相关设置也不会生效。论文称非 GRF 环境使用 VDN，因此应把“是否按预期使用 mixer”列为独立消融，而不要默认为配置名就等于实际算法。

7. 阶段 D：未知队友与头选择评测

评测必须将测试队友与其描述完全隔离于训练阶段。执行前只允许使用描述文本选择头，不能根据该测试队友的回报挑选最优头；后者对应论文中的事后上界，而不是 SemDiv 的真实部署设置。

一次标准评测应输出四类内容：每个测试队友的 R1、R2；LLM 所选头及其理由；所有候选头的离线表现矩阵；跨随机种子的均值和标准差。额外保留头选择错误案例：描述相近但应选择不同头、描述含糊、或 LLM 输出非法编号。它们比平均分数更能暴露语言路由的边界。

8. 建议的阶段验收清单

阶段	必须通过的证据	常见失败信号	下一步
基础环境	reset/step、动作掩码、原始回报均合理	维度不符、回合不终止、回报全零	回退到零额外奖励测试
人工奖励	单步触发与回合累计值一致	奖励从不触发或每步都触发	写最小状态单元测试
候选队友	原始回报和额外回报均有效	failed 或 constant	审查尺度与事件可达性
行为对齐	轨迹事件、规则判定、LLM 核验一致	misaligned	修改可观察行为定义
多样性	与历史队友的兼容性矩阵有可解释差异	similar	把相似信息用于下一轮 prompt
持续学习	新旧队友矩阵均记录且旧性能可接受	旧列整体下降	核查快照、冻结和正则
语言选择	所选头接近事后最佳头	选头经常错位	收集描述-选择-结果案例

最终汇总	R1/R2 均按种子与测试队友报告	只报最优运行	重做聚合并报告失败率
------	-------------------	--------	------------

六、关键实现细节与复现风险

1. 代码并非开箱即用

公开仓库没有完整锁定所有依赖版本，且其模型服务调用采用较早的 SDK 接口。复现前应在隔离环境中固定依赖版本，并通过安全的凭据管理机制配置模型服务；不得把访问凭据、服务地址或未公开实验配置写入公开文档与实验产物。

2. 运行方式的可移植性

原始自动化流程依赖特定操作系统的文件和进程管理工具。部署到其他系统前，应以跨平台实现替代相关操作，并单独验证子进程管理、日志采集和失败恢复；本笔记不绑定具体运行条件。

3. LLM 生成代码直接修改环境源码

实现会将生成的奖励函数接入环境逻辑；失败恢复依赖于实验过程中的备份。因此应在隔离副本中运行生成代码，并为每次尝试保存行为文本、提示、原始模型输出、奖励函数版本和随机种子，避免污染基线环境或混淆实验来源。

4. 论文叙述与代码实现需要分别验收

论文中“`lambda_2` 从 1 衰减到 0”是方法要求；仓库中实际衰减逻辑分散在不同环境的奖励 wrapper 和训练代码。复现时必须检查每环境是否真按预期记录/衰减额外奖励，不能仅看 `test_return_original_mean`。

此外，PyMARL MultiHeadLearner 目前将 `args.mixer = None`，即使配置文件填了 VDN/QMIX 也不启用 mixer；这与 README/论文“使用 VDN”的表述存在需要核对的实现差异。正式复现前应明确这是作者有意的独立 Q 学习设置、代码遗留，还是需要修正的 bug，并在实验记录中写明。

5. 成本、随机性与验证偏差

一次完整生成最多要得到 6 个有效队友；每个行为最多尝试 2 次、总行为尝试上限 30。LLM 既生成代码又做轨迹对齐和头选择，因此 API 模型、版本、temperature、重试记录都会显著影响结果。轨迹对齐还采用 5 次 LLM 投票，它是启发式验收而非确定性真值；对 LBF/PP/SMACv2 建议额外实现可程序化验证的行为事件指标，作为 LLM 判断的独立证据。

另一个容易被忽视的偏差来自“生成失败的选择性丢弃”：如果只报告最终成功的 6 个行为，读者看不到 LLM 提议了多少不可执行、无效或重复的奖励。建议同时报告候选行为总数、各失败类别占比、每个有效队友的平均尝试次数，以及从首个候选到收集满队友池的总训练消耗。这样才能评价 SemDiv 的“效率”而不只是最终性能。

七、建议记录的实验表

1. 运行总览记录

每次完整运行一行，记录确保别人能够判断“这个结果是否可比”：

字段	记录内容
experiment_id	实验卡编号，不复用
目标环境与任务版本	公开任务名称与版本标识
代码/依赖/模型版本	可公开的版本标识
随机种子集合	训练与评测种子，不可中途替换
有效队友数	目标数量、实际数量与达成时间
候选总数	成功、失败、恒定、失配、相似的计数
训练预算	队友、ego、评测各自的环境步数或更新次数
最终汇总	R1、R2 的均值、标准差和每项有效样本数
结论状态	smoke、method、partial-reproduction 或 result-reproduction

2. 候选行为记录

每个候选行为一行，至少记录：

字段	含义
behavior_id / 文本	LLM 提出的协作偏好
prompt 与模型版本	可重放的生成上下文
额外奖励代码 hash	防止同一描述对应不同代码而不可追踪
尝试次数与失败状态	bug / failed / constant / misaligned / similar
原始回报、额外回报	任务完成与行为优化必须同时检查
轨迹摘要与对齐投票	对齐证据
相似队友及相对回报	多样性证据
队友模型版本列表	论文使用初、中、末训练阶段的策略
ego 头编号、正则系数、旧任务成绩	持续学习与遗忘证据

测试队友上的 R1 / R2	最终泛化指标
----------------	--------

3. 每轮 ego 持续学习记录

字段	含义
round	当前第几个有效队友/策略头
当前行为与队友版本	本轮新学对象
共享特征快照	是否在训练前成功缓存
旧头冻结检查	旧头参数的变化范数应为零或符合设计
正则项统计	均值、末值及其与训练损失的比例
新队友表现	当前头与当前队友的 R1/R2 或回报
旧队友平均表现	所有历史队友上的均值与最差值
队友采样范围	加载/评测的检查点数与实际更新采样集合
兼容性矩阵位置	结果表或可视化的引用编号
结论	保持、轻微遗忘、严重遗忘或训练失败

4. 复现日志的写法

复现日志应区分“观测到的事实”和“解释”。例如：

Plaintext
事实：候选行为 B-07 的原始回报达到门槛，但额外回报在训练前后无明显变化； 三段轨迹均未出现目标事件。
解释：奖励事件可能不可达，或状态在奖励计算前尚未更新。
动作：先增加事件触发计数，再决定修改奖励条件还是替换行为描述。

这种写法比“本轮效果不好”更有价值，因为它保留了下一位复现者可以验证、反驳或继承的因果假设。对于 LLM 参与的系统，必须归档原始输出和解析后的奖励函数；仅保存最终自然语言总结，会丢失定位解析错误与模型漂移的证据。

建议把“公开可分享记录”与“受控访问原始记录”分开管理。前者只包含论文结论、去标识化的行为文本、聚合指标和公开代码版本；后者才保存完整运行日志和模型产物，并按团队数据治理要求访问。

八、我的理解与可进一步验证的问题

SemDiv 最有价值的不是“让 LLM 写奖励代码”本身，而是把难以搜索的策略空间先压缩为人可理解、可比较、可反馈的协作意图空间。语言在这里承担的是探索先验和策略路由接口：前者帮助发现多次传球这类稀有策略，后者让部署阶段能在零交互前选择头。

但这也带来两个需要单独验证的假设。第一，语言描述是否足够覆盖真实队友的行为差异？如果同一句偏好在不同状态下对应截然不同的策略，文本路由可能失效。第二，额外奖励是否真的“实现”了描述，而非仅利用环境统计相关性骗过轨迹检查？因此建议增加以下消融：固定人工奖励、去掉相似性反馈、随机头选择、规则式头选择、程序化对齐验收，以及测试描述改写/含糊化下的鲁棒性。

参考

[1] Li, Yuan, Liu, Jiang, Yu. LLM-Assisted Semantically Diverse Teammate Generation for Efficient Multi-agent Coordination. ICML 2025.

[2] SemDiv 公开代码仓库说明。

[3] 官方仓库：<https://github.com/lilh76/SemDiv>